

AI Career Companion

Full Technical Documentation

An AI-powered, full-stack career platform using Retrieval-Augmented Generation (RAG), semantic vector search, and large language models to help students find internships and prepare for interviews.

Report by: Narenkumar C
Mentor: Vijay Pawar

Table of Contents

Table of Contents	2
1. Product Overview	4
2. System Architecture	5
3. Features & Functionality.....	6
3.1 Secure Authentication.....	6
3.2 Resume Upload & Parsing.....	6
3.3 RAG Internship Matching.....	7
3.4 InternAI Chatbot (RAG + Memory).....	7
3.5 Interview Preparation Agent	7
3.6 Document Q&A	7
3.7 ATS Score Analyser	8
3.8 Internship Catalog	8
4. How It Works — Detailed Flows.....	8
Authentication Flow	8
Resume Upload & Parsing Flow	8
RAG Internship Matching Flow	8
InternAI Chatbot Flow	9
Interview Agent Flow	9
Document Q&A Flow	9
5. RAG Flow Diagram	10
6. Conversation Memory	11
7. Database Schema	12
users.....	12
refresh_tokens	12
resumes	12
chat_sessions	13
chat_messages	13
8. API Reference.....	14
Full Endpoint Table.....	14
9. Project Structure	15
10. Tech Stack.....	16
11. Setup & Running.....	17
Environment Variables (.env)	17

Step-by-Step	17
12. Sample Outputs	18
Internship Match Response	18
Parsed Resume Response	18
Chatbot Conversation (Memory Demo)	18
13. Key Design Decisions	19

1. Product Overview

Product Name: AI Career Companion (InternAI)

Description:

An AI-powered, full-stack career platform that helps university students and early-career professionals find the right internships and prepare for job interviews. The platform uses Retrieval-Augmented Generation (RAG), semantic vector search, and large language models to automate and personalise every step of the job-search journey.

Problem solved:

Students spend hours manually scanning job boards, tailoring resumes, and guessing at interview questions. InternAI automates resume parsing, surfaces semantically matched internships, and provides a resume-aware AI coach that generates personalised interview content on demand.

Target users: University students and early-career professionals seeking internships.

Main objectives:

- Automate resume parsing (regex + LLM hybrid)
- Provide accurate semantic internship recommendations via RAG
- Maintain contextual conversation history for the chatbot
- Offer personalised interview preparation using the candidate's own resume
- Support document Q&A for any uploaded PDF/DOCX

2. System Architecture

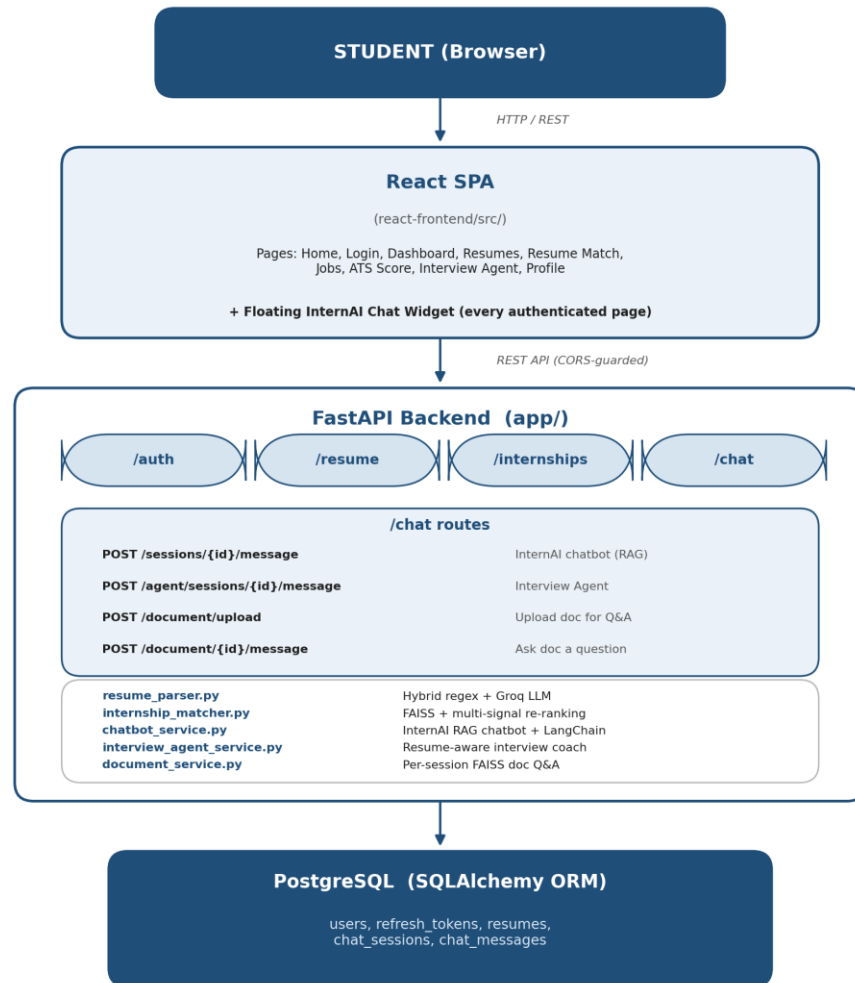


Figure 2.1 — System architecture and request flow

Component Responsibilities

Component	Responsibility
React SPA	User interface: auth, resume upload, matching, chat, interview prep
FastAPI	Business logic, request validation, DB interactions, JWT auth
LangChain + Groq LLM	Prompt composition, conversation chains, AI response generation
FAISS (internship index)	Pre-built vector store for semantic internship matching
FAISS (chatbot index)	Vector store over product knowledge doc for chatbot RAG
FAISS (doc sessions)	Per-upload temporary vector store for Document Q&A
PostgreSQL	Persistent storage: users, resumes (JSONB fields), chat history

3. Features & Functionality

3.1 Secure Authentication

- JWT access token (30 min) + refresh token (7 days, DB-stored)
- bcrypt password hashing; plain-text password never stored
- Token rotation: using a refresh token issues a new pair and revokes the old one
- Logout = mark refresh token as revoked in DB
- Password reset via time-limited token
- Account enumeration prevention: identical errors for wrong email / wrong password

3.2 Resume Upload & Parsing

Users upload a PDF or DOCX resume (≤ 5 MB).

Extraction pipeline:

- Stage 1 — Regex: fast, deterministic extraction of email, phone, LinkedIn URL, GitHub URL
- Stage 2 — LLM (Groq): contextual extraction of skills, technical_skills, soft_skills, education, work_experience, projects, certifications, internships, languages, achievements, professional_summary
- Merge: regex fields take priority for contact info; LLM fills everything else
- Parsed fields stored as JSONB in PostgreSQL; parsed_status = "parsed" | "failed"

3.3 RAG Internship Matching

Semantic search over ~100+ pre-embedded internship postings.

Matching signals and weights

Signal	Weight	How calculated
Skill overlap	50%	matched_skills / total required_skills
Semantic similarity	20%	FAISS cosine similarity
Education match	15%	degree level vs. posting min_education
Location/mode	15%	city name match; Remote = 1.0 always

Match labels

Score	Label
>= 85%	Perfect Match
>= 65%	Strong Match
>= 40%	Partial Match
< 40%	Weak Match

3.4 InternAI Chatbot (RAG + Memory)

A floating widget on every authenticated page. Answers questions about the product using the product knowledge DOCX as its knowledge base. Maintains full conversation memory per session in PostgreSQL.

3.5 Interview Preparation Agent

An AI coach on the Interview Agent page. Reads the candidate's parsed resume from the DB and uses it as personalised context for:

- Role and company recommendations
- Technical, behavioral, and situational interview questions
- Model answers referencing the candidate's own projects
- Time-bound preparation roadmaps
- Learning path and resource recommendations
- Skill gap analysis

3.6 Document Q&A

Upload any PDF or DOCX (up to 10 MB). The system:

- Extracts text (PyMuPDF / python-docx)
- Chunks the text (600 chars, 80 overlap)
- Embeds chunks with all-MiniLM-L6-v2
- Stores in a per-session FAISS store
- Answers user questions using top-5 retrieved chunks + Groq

3.7 ATS Score Analyser

Submit a resume and a job description; the system returns an ATS compatibility score and suggestions for improvement.

3.8 Internship Catalog

Public endpoint (no auth required). Browse all postings; filter by domain, location, and mode.

4. How It Works — Detailed Flows

Authentication Flow

```
POST /auth/register --> bcrypt hash --> INSERT users row
POST /auth/login --> verify hash --> access token (30 min) + refresh token (7 days)
Protected request --> Bearer <access_token> --> JWT decode & validate
POST /auth/refresh --> revoke old refresh token --> issue new pair
POST /auth/logout --> mark refresh token revoked in DB
```

Resume Upload & Parsing Flow

```
POST /resume/upload
1. Validate type (pdf/docx) and size (<= 5 MB)
2. Save to disk: uploads/resumes/<user_id>/<uuid>.<ext>
3. Insert DB row (parsed_status = "pending")
4. Resume parser:
  - Extract raw text (PyMuPDF / python-docx)
  - REGEX PASS: email, phone, linkedin_url, github_url
  - LLM PASS (Groq): skills, technical_skills, soft_skills,
                    education, work_experience, projects,
                    certifications, internships, languages,
                    achievements, professional_summary
  - MERGE: regex fields take priority
  - UPDATE resume row (parsed_status = "parsed")
  - On failure: parsed_status = "failed" (upload still succeeds)
```

RAG Internship Matching Flow

```
GET /internships/match/{resume_id}?k=5
1. Build query text: skills + education + summary + experience + projects
2. Embed with all-MiniLM-L6-v2 (384-dim)
```

3. FAISS search: pull 4x k candidates (wider pool for re-ranking)
4. Multi-signal re-ranking per candidate (50/20/15/15 weights)
5. Sort by composite score, return top k
6. Groq LLM summary (2-3 sentences on fit + 1-2 skill suggestions)

InternAI Chatbot Flow

POST /chat/sessions/{id}/message

1. Load last 6 messages from chat_messages (DB) --> LangChain history
2. RAG: top-3 chunks from faiss_chatbot_index
3. Compose: System (InternAI persona + RAG context) + History + Query
4. Invoke Groq --> response
5. Store user message + assistant message in chat_messages

Interview Agent Flow

POST /chat/agent/sessions/{id}/message

1. Load parsed resume from DB --> format as structured text context
2. Load last 8 messages from chat_messages --> LangChain history
3. Compose: System (InterviewGPT persona + resume context) + History + Query
4. Invoke Groq --> response
5. Store both messages in chat_messages

Document Q&A Flow

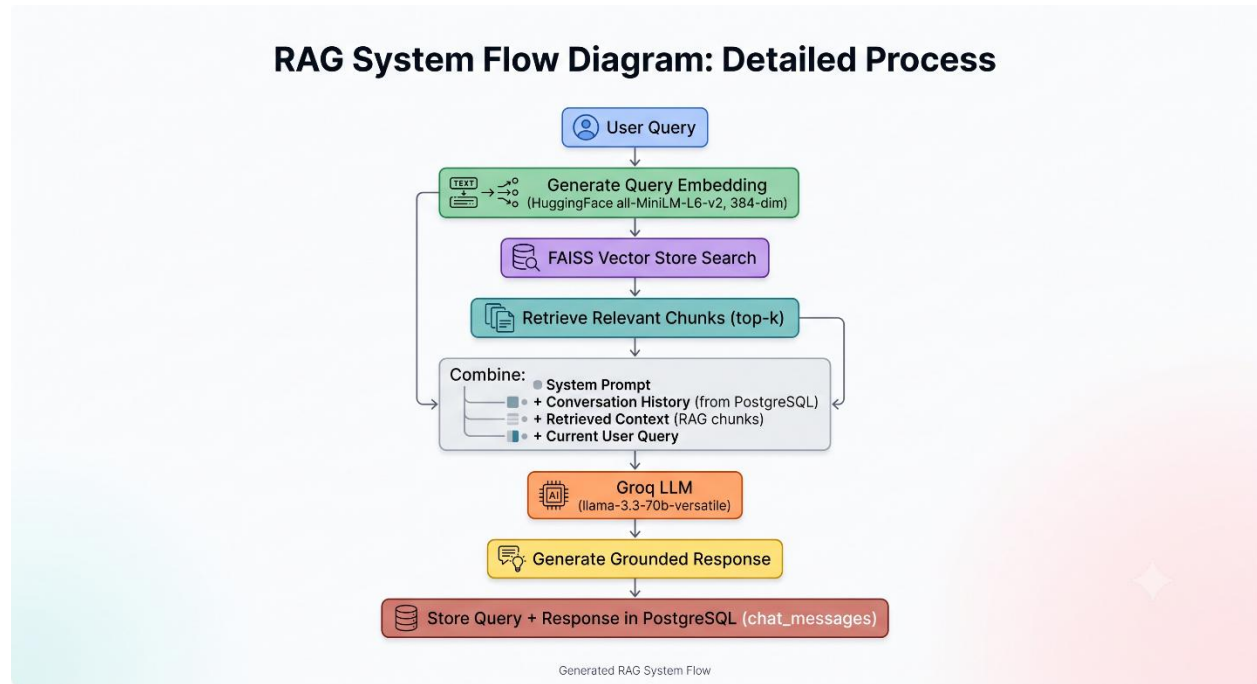
POST /chat/document/upload

1. Validate type and size (<= 10 MB)
2. Extract text (PyMuPDF / python-docx)
3. Chunk (RecursiveCharacterTextSplitter: 600 chars, 80 overlap)
4. Embed with all-MiniLM-L6-v2
5. Save per-session FAISS store --> return doc_session_id

POST /chat/document/{doc_session_id}/message

1. Retrieve top-5 chunks from session FAISS store
2. Compose: Document Context + User Question
3. Invoke Groq --> markdown-formatted answer

5. RAG Flow Diagram



Sprint Overview

The project was developed across **5 sprints**, with each sprint delivering a distinct layer of the platform.

Sprint	Focus Area	Key Deliverables
Sprint 1	Resume Parsing (Regex + LLM)	Hybrid extraction pipeline using deterministic regex for contact fields (email, phone, LinkedIn, GitHub) and Groq LLM for free-form sections (skills, education, experience, projects, certifications). Merged output stored as JSONB in PostgreSQL.
Sprint 2	Backend Development	Full FastAPI backend with SQLAlchemy ORM, JWT authentication (access + refresh tokens), bcrypt password hashing, token rotation, PostgreSQL schema, REST endpoints (/auth, /resume, /internships, /chat), and FAISS internship index build pipeline.
Sprint 3	Frontend Creation	React 18 + TypeScript + Vite SPA with 9 pages: Home, Login, Dashboard, Resumes, Resume Match, Jobs, ATS Score, Interview Agent, and Profile. Implemented TailwindCSS dark-mode design, AppShell sidebar, and authentication state management using Context API.
Sprint 4	Resume Matching + Chatbot Integration	FAISS-based semantic internship matching with multi-signal re-ranking: skill overlap (50%), semantic similarity (20%), education (15%), and location (15%). Integrated the floating InternAI chatbot using RAG over a product knowledge document with per-session PostgreSQL conversation memory.

Sprint 5	Interview Preparation Agent	Resume-aware AI coach (<code>/chat/agent/*</code>) that reads the candidate's parsed resume from the database and generates personalised interview questions, model answers referencing their projects, skill-gap analysis, learning roadmaps, and time-bound preparation plans using LangChain + Groq.
-----------------	------------------------------------	---

6. Conversation Memory

The chatbot and interview agent maintain session-scoped memory via PostgreSQL:

- Each user has multiple chat sessions (chat_sessions table)
- Each session stores ordered messages (chat_messages table)
- On every request: last 6 (chatbot) or 8 (interview agent) messages are loaded, converted to LangChain HumanMessage / AIMessage objects, and injected into the prompt via MessagesPlaceholder

User & Session Management

```
User A (user_id)
+-- Session 1 (session_id)
| +-- Message 1: role=user,  message="What technologies are used?"
| +-- Message 2: role=assistant, message="Python, FastAPI, LangChain..."
| +-- Message 3: role=user,  message="Why did we use it?"
| +-- Message 4: role=assistant, message="FastAPI was chosen for..."
+-- Session 2 (session_id)
+-- ...
```

Conversations from different users or sessions are never mixed.

7. Database Schema

users

Column	Type	Notes
id	UUID (PK)	auto-generated
full_name	VARCHAR(150)	
email	VARCHAR(255)	unique, indexed
hashed_password	VARCHAR(255)	bcrypt
is_active	BOOLEAN	default true
reset_token	VARCHAR(512)	nullable
created_at	TIMESTAMPTZ	

refresh_tokens

Column	Type	Notes
id	UUID (PK)	
user_id	UUID (FK -> users)	CASCADE delete
token	VARCHAR(512)	unique, indexed
expires_at	TIMESTAMPTZ	
revoked	BOOLEAN	default false

resumes

Column	Type	Notes
id	UUID (PK)	
user_id	UUID (FK -> users)	CASCADE delete
original_filename	VARCHAR(255)	
stored_filename	VARCHAR(255)	random UUID-based
file_path	VARCHAR(500)	server filesystem path
content_type	VARCHAR(100)	
file_size	INTEGER	bytes
parsed_status	VARCHAR(20)	pending / parsed / failed

Column	Type	Notes
full_name	VARCHAR(150)	regex
email	VARCHAR(150)	regex
phone	VARCHAR(30)	regex
linkedin_url	VARCHAR(255)	regex
github_url	VARCHAR(255)	regex
professional_summary	TEXT	LLM
skills	JSONB	LLM
technical_skills	JSONB	LLM
soft_skills	JSONB	LLM
education	JSONB	LLM
work_experience	JSONB	LLM
projects	JSONB	LLM
certifications	JSONB	LLM
internships	JSONB	LLM
languages	JSONB	LLM
achievements	JSONB	LLM

chat_sessions

Column	Type	Notes
id	UUID (PK)	
user_id	UUID (FK -> users)	CASCADE delete
created_at	TIMESTAMPTZ	

chat_messages

Column	Type	Notes
id	UUID (PK)	
session_id	UUID (FK -> chat_sessions)	CASCADE delete
role	VARCHAR(50)	"user" or "assistant"
message	TEXT	
created_at	TIMESTAMPTZ	

8. API Reference

Authentication — POST /auth/register

Request:

```
{ "full_name": "Naren Kumar", "email": "naren@example.com", "password": "secret123" }
```

Authentication — POST /auth/login

Request: { "email": "...", "password": "..." }

Response: { "access_token": "...", "refresh_token": "...", "token_type": "bearer" }

Full Endpoint Table

Auth (/auth)

Method	Endpoint	Auth	Description
POST	/auth/register	-	Create account
POST	/auth/login	-	Login; returns token pair
GET	/auth/me	JWT	Current user profile
POST	/auth/refresh	-	Rotate refresh token
POST	/auth/logout	-	Revoke refresh token
POST	/auth/forgot-password	-	Send reset token
POST	/auth/reset-password	-	Reset with token
POST	/auth/change-password	JWT	Change while logged in

Resume (/resume)

Method	Endpoint	Auth	Description
POST	/resume/upload	JWT	Upload PDF/DOCX, trigger parse
GET	/resume/	JWT	List all resumes + parsed data
GET	/resume/{id}/download	JWT	Download original file

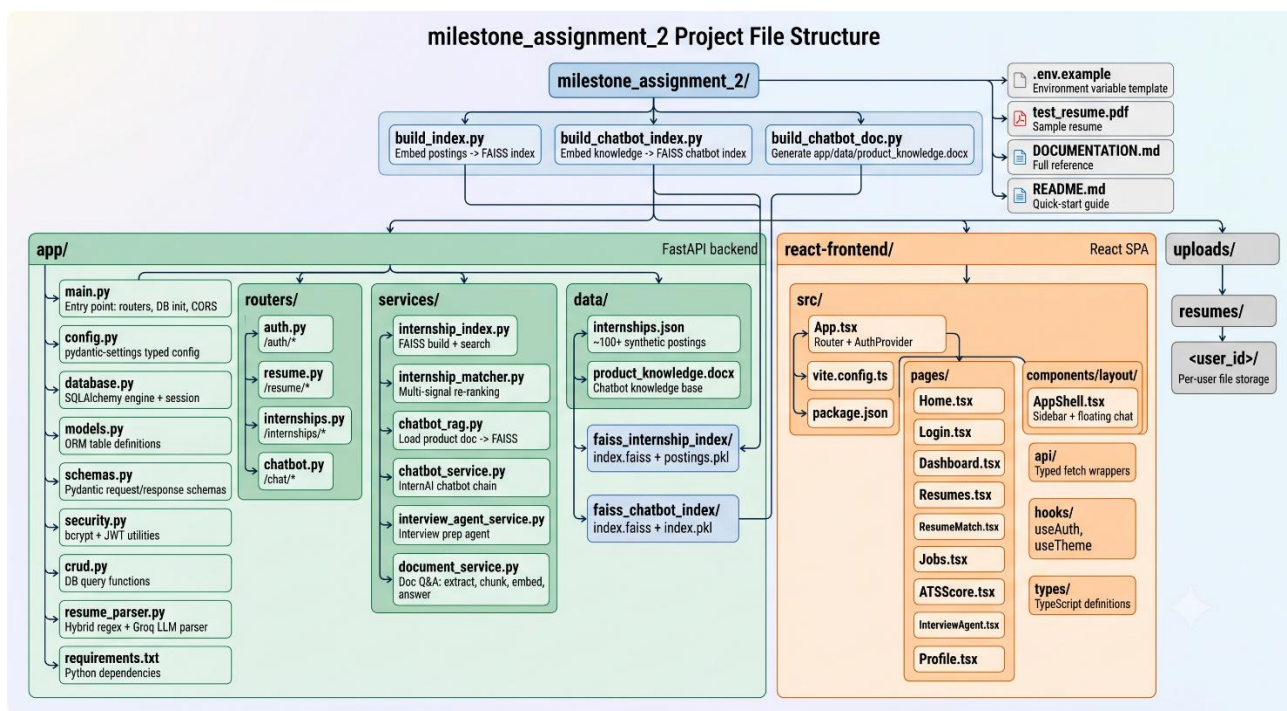
Internships (/internships)

Method	Endpoint	Auth	Description
GET	/internships/	-	Browse catalog (public)
GET	/internships/match/{resume_id}?k=5	JWT	RAG match + re-rank + summary

Chatbot & Agents (/chat)

Method	Endpoint	Auth	Description
POST	/chat/sessions	JWT	Create InternAI chat session
GET	/chat/sessions	JWT	List chat sessions
POST	/chat/sessions/{id}/message	JWT	Message the InternAI chatbot
POST	/chat/agent/sessions	JWT	Create interview agent session
GET	/chat/agent/sessions	JWT	List agent sessions
POST	/chat/agent/sessions/{id}/message	JWT	Message the Interview Agent
POST	/chat/document/upload	JWT	Upload doc for Q&A (<=10 MB)
POST	/chat/document/{doc_id}/message	JWT	Ask question about doc

9. Project Structure



10. Tech Stack

Layer	Technology	Why chosen
API	FastAPI (Python 3.10+)	High-performance async; auto OpenAPI docs; native pydantic integration
Frontend	React 18, TypeScript, Vite	Type safety, fast HMR, production-grade SPA routing
Styling	TailwindCSS	Utility-first; dark-mode trivial; consistent design tokens
LLM Framework	LangChain	Clean chain/prompt composition; MessagesPlaceholder for history injection
LLM	Groq llama-3.3-70b-versatile	Very fast inference; handles parsing, chat, interview, doc Q&A
Embeddings	HuggingFace all-MiniLM-L6-v2	CPU-only, no API key; 384-dim; fast and accurate for semantic search
Vector Search	FAISS IndexFlatIP	Lightweight, in-process; no external service needed
Database	PostgreSQL	JSONB for flexible resume fields; robust relational storage
ORM	SQLAlchemy 2.0	Type-safe queries; auto table creation
Auth	JWT HS256 + bcrypt	Industry-standard; refresh token rotation via DB
File Parsing	PyMuPDF, python-docx	Reliable text extraction from PDF and DOCX
Settings	pydantic-settings	Typed .env config; validated at startup

11. Setup & Running

Environment Variables (.env)

```
DATABASE_URL=postgresql+psycopg://postgres:yourpassword@localhost:5432/ai_internship_agent
SECRET_KEY=your-random-secret-key-at-least-32-chars
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7
RESET_TOKEN_EXPIRE_MINUTES=15
GROQ_API_KEY=your-groq-api-key
```

Step-by-Step

```
# 1. Python virtualenv
python -m venv myenv
myenv\Scripts\activate # Windows
pip install -r app/requirements.txt

# 2. PostgreSQL
# Run in psql: CREATE DATABASE ai_internship_agent;

# 3. Build FAISS indices
python build_index.py # internship matching index
python build_chatbot_doc.py # product knowledge DOCX
python build_chatbot_index.py # chatbot RAG index

# 4. Start backend
uvicorn app.main:app --reload
# http://127.0.0.1:8000/docs

# 5. Start React frontend (separate terminal)
cd react-frontend
npm install
npm run dev
# http://localhost:5173
```

12. Sample Outputs

Internship Match Response

```
{
  "resume_id": "a1b2c3d4-...",
  "query_skills": "Python, FastAPI, PostgreSQL, Docker",
  "summary": "Strong backend alignment with 3 of 5 matches. Adding React/TypeScript would open 40% more full-stack roles.",
  "results": [{
    "role_title": "Backend Developer Intern",
    "company": "TechCorp",
    "match_percentage": 87.3,
    "match_label": "Perfect Match",
    "matched_skills": ["Python", "FastAPI", "PostgreSQL"],
    "missing_skills": [],
    "skill_score": 1.0,
    "semantic_score": 0.82,
    "education_score": 1.0,
    "location_score": 1.0
  }]
}
```

Parsed Resume Response

```
{
  "parsed_status": "parsed",
  "full_name": "Naren Kumar",
  "email": "naren@example.com",
  "technical_skills": ["Python", "FastAPI", "React", "PostgreSQL"],
  "soft_skills": ["Communication", "Teamwork"],
  "education": [{"institution": "Shiv Nadar University Chennai", "degree": "B.Tech CSE (IoT)"}],
  "work_experience": [{"title": "SDE Intern", "company": "Infosys"}],
  "projects": [{"name": "ReWear", "description": "Second-hand clothing marketplace"}],
  "languages": ["English", "Tamil"]
}
```

Chatbot Conversation (Memory Demo)

```
User:  "What technologies are used in this product?"
Assistant: "Python + FastAPI backend, React + TypeScript frontend,
           LangChain + Groq for AI, FAISS for vector search,
           PostgreSQL for storage."

User:  "Why did we use it?"
Assistant: "FastAPI was selected for high performance and async support.
           LangChain cleanly composes prompts with history..."
```

The chatbot resolves "it" correctly from conversation history stored in PostgreSQL.

13. Key Design Decisions

Decision	Rationale
Regex + LLM hybrid parser	Regex is deterministic for contact fields; LLM handles free-form sections that vary wildly across resumes
FAISS pre-built internship index	Embedding 100+ postings per request is too slow; pre-build + cache = millisecond queries
4x candidate pool before re-ranking	Pure cosine similarity is weak alone; wider pool lets skill/education/location signals refine results
Skills weighted 50%	Skill match is the dominant hiring signal for internships
JSONB for resume fields	Open-ended structure avoids schema churn; still queryable in Postgres
UUID primary keys	Unguessable IDs prevent enumeration attacks
Refresh token DB storage	JWTs cannot be revoked client-side; DB row enables logout + rotation
Session-scoped conversation memory	Last 6-8 messages injected per request via MessagesPlaceholder; separate per user+session
Per-session FAISS for doc Q&A	UUID-keyed per-upload store prevents cross-user data leakage
Resume context in interview agent	DB-loaded parsed fields make responses specific, not generic
LangChain chain composition	ChatPromptTemplate + MessagesPlaceholder = clean, readable prompt assembly
React + Vite SPA	Production-grade frontend with proper auth state, routing, and a floating chat widget